TITLE AND DATE

Document name: Project#3 Asynchronous Execution Lab Report
Document reference: kiddKid.cmpe311.fall26.project#3
Date of publication: October 3, 2026

LEAD ENGINEER: William Wang

STAKEHOLDERS:

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA
MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

HIGH-LEVEL DESCRIPTION: This document is the project document for Project#1 of the CMPE311 class. It contains customer, technical and testing requirements as well as the design and results of the validation testing.

DESCRIPTION: This design is to create an embedded system based upon an Arduino Uno R3 development platform that allows the user of the program to specify an LED number and its blink interval. The blinking of the LEDs must continue asynchronously without any additional input from the user. Included in this document are the customer requirements obtained from the customer, the high-level technical requirements derived from those customer requirements, the design, the testing scenarios and requirements, and the results of testing. Appended to this document is the code executed and a video of those tests that required video documentation.

RESULT SUMMARY: The project was a success, the embedded system design meets all testing and high-level requirements.

REFERENCES AND GLOSSARY

REFERENCES:

- ProjectAsynTasking_draft – The definition of the project this document addresses
- CMPE310 Project #5 – A similar project assigned in CMPE310 in Spring 2025
- Arduino UNO R3 Product Reference Manual SKU A000066, 12/03/2024
- Async Programming in Arduino: Unleashing the Power of Non-Blocking Code, Mahdi Valizadeh, medium.com, 4/8/2024

***DEFINITIONS*:**

"The User" – The person operating (not programming) the embedded system

"The System" – The embedded system being operated by The User

"The Customer" – The person(s) paying for the embedded system being designed and built

"The Developer" – The person(s) designing and building the System

"The Evaluator" – The person(s) that determine whether or not The System satisfies The Customer-requirements.

"The Customer-requirements" – The requirements defined by The Customer as satisfying The Contract.

"The Requirements" – The System's high-level technical requirements derived from The Customer-requirements.

"The Educational-constraints" – Requirements imposed by the instructor unrelated to the embedded system that allow The System to be evaluated.

"The Company" – The organization The Customer has contracted with to build The System.

"The Contract" – The business document that legally binds The Company to provide some service or product to The Customer.

"serial-monitor" – The serial port used by the Arduino IDE to communicate with The User.

"The Reference-platform" – The configuration of The System used by The Developer to test and validate

"PROJECT-ASYNC", "PROJECT-CE" – The previous projects upon which this project is based

"arduino.h" - helper functions for the Arduino development platform maintained by Arduino AG, owned by Qualcomm.

IP – Intellectual Property

ISR – Interrupt Service Routine

PWM – Pulse Width Modulation

The System. For this class, The System is the Arduino compatible ELEGOO Uno R3 development board.

ACRONYMS AND ABBREVIATIONS:

Arduino – an Italian open-source hardware and software company; also refers to a development board created by the company

arduino.h – header for a library of convenience functions specific to the Arduino development platform

AVR – A family of microcontrollers, originally developed by Atmel, and currently owned by Microchip Technology

ELEGOO – A Chinese company that develops and markets 3D printers and accessories

IDE – Integrated Development Environment

gcc – front end for the GNU Compiler Collection

Github – A widely used distributed SVC (Software Version Control) system

LED – Light Emitting Diode

REQUIREMENTS (Part I)

CONVENTIONS:

Must, shall or will – your design must satisfy the requirement
May – your design may satisfy the requirement but doesn't have to
Informative – the intent of the following description is to make the requirement more understandable
All customer requirements are started with "C.#".
All high-level technical requirements are started with "HL.#".
All testing/validation requirements are started with "T.#"

CUSTOMER REQUIREMENTS:

The customer requirements, C1 through C6, are identical to that of PROJECT-ASYNC. See CUSTOMER REQUIREMENTS section.

HIGH-LEVEL TECHNICAL REQUIREMENTS:

The high-level technical requirements, HL.1 through HL.5, are identical to that of PROJECT-ASYNC. See HIGH-LEVEL TECHNICAL REQUIREMENTS section.

REQUIREMENTS (Part II)

HIGH-LEVEL TECHNICAL REQUIREMENTS:

The high-level technical requirements, HL.1 through HL.6, are identical to that of PROJECT-ASYNC. See HIGH-LEVEL TECHNICAL REQUIREMENTS section.

EDUCATIONAL RESTRAINTS REQUIREMENTS:

EC.1.0 You must replace the milli() funcIon provided by arduino.h with your own. Your replacement must be named mymilli(). Outside of replacing the functionality, you may extend its capability beyond that of milli().

EC.1.0.1 You must write your own PWM counter ISR using it to provide the Iming mechanism underlying mymilli().

EC.1.0.2 You must define your own register constants that would normally be defined in avr.h. These constants do not need to have the same names as those in avr.h.

EC.1.1 The system tesIng and validaIon requirements must contain those defined in PROJECT-CE.

EC.1.1.1 The tesIng and validaIon requirements MUST be extended to contain any additional tests necessary to properly evaluate/demonstrate the funcIon of the modified system.

EC.1.2 The final report must be a new revision of the formal design document produced in PROJECT-CE. Make sure you note any changes you've made to the design.

EC.1.3 You must provide a properly formatted ECN notifying other teams of the change

DESIGN:

DESIGN PRE-REQUISITES:

1. ELEGOO Arduino Uno R3 clone
2. Arduino IDE 2.3.3 or better

DEVELOPMENT PLATFORM:

1. See DESIGN PRE-REQUISITE 2 above

ANY ADDITIONAL DESIGN CONSIDERATIONS:

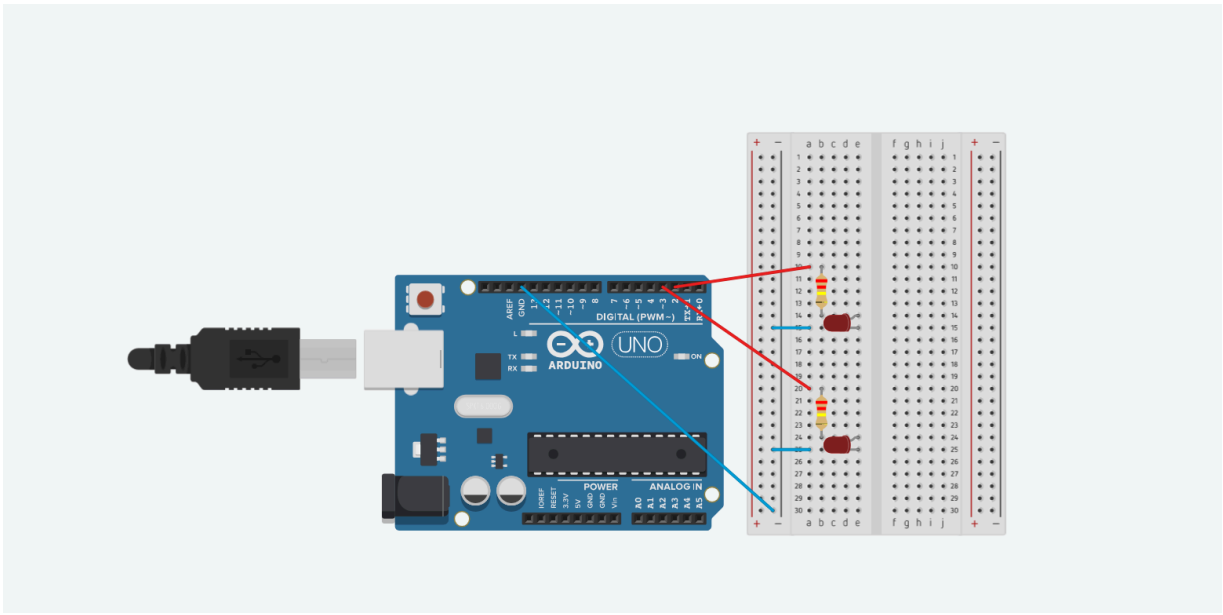
1. The program uses a round-robin cyclic executive task manager with a function pointer array

See Appendix A for the logical flow of the project program.
See Appendix B for the code executing on the Arduino Uno R3.

TESTING AND VALIDATION

DESCRIPTION: The tests that have to be performed and validated are shown in Table 2. These tests were performed on the testbed shown in Figure 1. Table 1 shows a dialog that must be successfully performed by the embedded system though testing must be more thorough than this. The results of testing are shown in Table 3. When necessary, a video of the test is provided along with this report.

Figure 1. Testbed Setup. LEDs connected to digital pins 2 & 3.



TESTING PLATFORM:

1. ELEGOO Arduino Uno R3 clone
2. Arduino IDE 2.3.3 or better
3. Power: Testing platform powered through USB cable, LEDs connected directly to Digital Outputs

Table 1. Required Test Dialog (***blue*** are the user inputs).

Serial port I/O	**Notes**
What LED? (1 or 2) ***2***	
What interval (in msec)? ***600***	LED2 starts blinking at an interval of 300ms on and 30,0ms off. LED1 is unaffected.

What LED? (1 or 2) ***1***	
What interval (in msec)? ***1600***	LED1 starts blinking at an interval of 8800ms on and 800ms off. LED2 is unaffected.

TESTING AND VALIDATION REQUIREMENTS:

Table 2. Testing and Validation Requirements

The testing and validation requirements, T.0 through T.4.1, are identical to that of PROJECT-ASYNC. See TESTING AND VALIDATION REQUIREMENTS section. T.5 The cyclic executive must execute each task in round-robin order. T.6 Serial input must not prevent either LED task from continuing to execute. T.7 Both LEDs must continue to operate correctly while being managed by the cyclic executive. T.8 Anytime timing is required, mymillis() is the only function that can be called to calculate timing. T.9 mymillis() should not hinder the program in anyway that is not expected.

TESTING RESULTS:

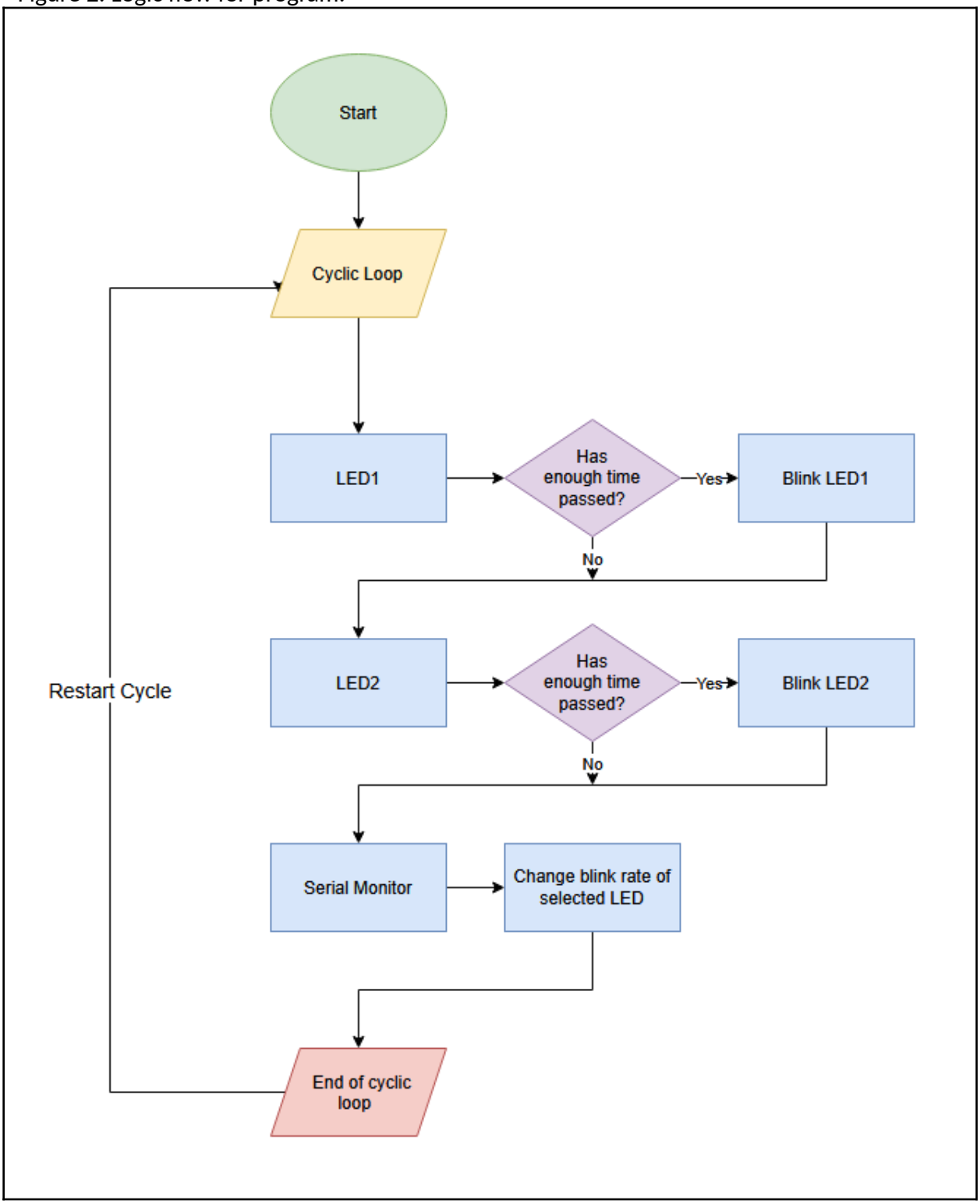
Table 3. Results of tests

Test performed	Results
T.0	Satisfied. See video of test.
T.1	Satisfied. See video of test.
T.1.A	Satisfied. See video of test.
T.2	Satisfied. See video of test.
T.3	Satisfied. See video of test.
T.4	Satisfied. See video of test.
T.4.1	Satisfied. See video of test.
T.5	Satisfied. See video of test.
T.6	Satisfied. See video of test.

T.7	Satisfied. See video of test.
T.8	Satisfied. See video of test.
T.9	Satisfied. See video of test.

Appendix A. Logic Flow Chart

Figure 2. Logic flow for program.



APPENDIX B. DESIGN CODE

```
#include <Arduino.h>
#include <limits.h>

/* Custom register definitions for the ATmega328P */
#define MY_TCCR2A (*(volatile unsigned char *)0xB0)
#define MY_TCCR2B (*(volatile unsigned char *)0xB1)
#define MY_TCNT2  (*(volatile unsigned char *)0xB2)
#define MY_OCR2A  (*(volatile unsigned char *)0xB3)
#define MY_TIMSK2 (*(volatile unsigned char *)0x70)
#define MY_TIFR2  (*(volatile unsigned char *)0x37)
#define MY_SREG   (*(volatile unsigned char *)0x5F)

volatile unsigned long myMilliCounter = 0;

int selectedLED = -1;

unsigned long blinkRateOne = 0;
unsigned long blinkRateTwo = 0;

unsigned long blinkTrackerOne = 0;
unsigned long blinkTrackerTwo = 0;

unsigned long inputValue = 0;
int inputStarted = 0;
int inputInvalid = 0;

/* Custom Timer2 compare-match interrupt */
extern "C" void __vector_7(void)
    __attribute__((signal, used, externally_visible));

void __vector_7(void)
{
  myMilliCounter++;
}

/* Returns a copy of the millisecond */
```

```
unsigned long myMilli(void)
{
  unsigned char savedStatus;
  unsigned long currentTime;

  savedStatus = MY_SREG;
  __asm__ __volatile__("cli" ::: "memory");

  currentTime = myMilliCounter;

  MY_SREG = savedStatus;
  __asm__ __volatile__("" ::: "memory");

  return currentTime;
}

/* Configures Timer2 to interrupt once every millisecond */
void setupMyMilli(void)
{
  unsigned char savedStatus;

  savedStatus = MY_SREG;
  __asm__ __volatile__("cli" ::: "memory");

  MY_TCCR2B = 0;        /* Stop Timer2 */
  MY_TIMSK2 = 0;        /* Disable Timer2 interrupts */
  MY_TCCR2A = 0x02;     /* Clear timer on compare match */
  MY_TCNT2 = 0;
  MY_OCR2A = 249;       /* 250 counts per interrupt */
  MY_TIFR2 = 0x07;      /* Clear old interrupt flags */

  myMilliCounter = 0;

  MY_TIMSK2 = 0x02;     /* Enable compare-match A interrupt */
  MY_TCCR2B = 0x04;     /* Start Timer2 with prescaler 64 */
```

```
  MY_SREG = savedStatus;
  __asm__ __volatile__("" ::: "memory");
}

/* Processes a complete number after Enter is pressed */
void processInput(void)
{
  unsigned long currentTime;

  if (inputInvalid) {
    Serial.println("Invalid input. Enter a whole number.");
  }
  else if (selectedLED == -1) {
    if (inputValue == 1 || inputValue == 2) {
      selectedLED = (int)inputValue;
    }
    else {
      Serial.println("Please choose LED 1 or 2.");
    }
  }
  else {
    currentTime = myMilli();

    if (selectedLED == 1) {
      blinkRateOne = inputValue;
      blinkTrackerOne = currentTime;
      digitalWrite(2, LOW);
    }
    else {
      blinkRateTwo = inputValue;
      blinkTrackerTwo = currentTime;
      digitalWrite(3, LOW);
    }

    selectedLED = -1;
  }
```

```
  inputValue = 0;
  inputStarted = 0;
  inputInvalid = 0;

  if (selectedLED == -1) {
    Serial.println("What LED? (1 or 2):");
  }
  else {
    Serial.println("What interval (in msec):");
  }
}

/* Reads one character at a time so LEDs keep blinking */
void taskSerial(void)
{
  char character;
  unsigned long digit;

  if (Serial.available() <= 0) {
    return;
  }

  character = (char)Serial.read();

  if (character == '\r' || character == '\n') {
    if (inputStarted) {
      processInput();
    }
  }
  else if (character >= '0' && character <= '9') {
    inputStarted = 1;
    digit = (unsigned long)(character - '0');

    /* Prevent the input number from overflowing */
    if (inputValue > (ULONG_MAX - digit) / 10UL) {
```

```
      inputInvalid = 1;
    }
    else {
      inputValue = inputValue * 10UL + digit;
    }
  }
  else {
    inputStarted = 1;
    inputInvalid = 1;
  }
}

void setup(void)
{
  Serial.begin(9600);

  pinMode(2, OUTPUT);
  pinMode(3, OUTPUT);

  digitalWrite(2, LOW);
  digitalWrite(3, LOW);

  setupMyMilli();

  Serial.println("What LED? (1 or 2):");
}

void loop(void)
{
  unsigned long currentTime;
  unsigned long halfInterval;

  currentTime = myMilli();

  /* LED 1 */
  if (blinkRateOne > 0) {
```

```
    halfInterval = blinkRateOne / 2;

    if (halfInterval == 0) {
      halfInterval = 1;
    }

    if (currentTime - blinkTrackerOne >= halfInterval) {
      digitalWrite(2, !digitalRead(2));
      blinkTrackerOne = currentTime;
    }
  }

  /* LED 2 */
  if (blinkRateTwo > 0) {
    halfInterval = blinkRateTwo / 2;

    if (halfInterval == 0) {
      halfInterval = 1;
    }

    if (currentTime - blinkTrackerTwo >= halfInterval) {
      digitalWrite(3, !digitalRead(3));
      blinkTrackerTwo = currentTime;
    }
  }

  taskSerial();
}
```

END OF DOCUMENT